Überprüfen des Lösungsdesigns

BeeGFS on NetApp with E-Series Storage

NetApp
March 21, 2024

This PDF was generated from https://docs.netapp.com/de-de/beegfs/beegfs-design-overview.html on March 21, 2024. Always check docs.netapp.com for the latest.

Inhalt

Überprüfen des Lösungsdesigns . . . 1
Designübersicht . . . 1
Hardwarekonfiguration . . . 1
Softwarekonfiguration . . . 2
Design-Überprüfung . . . 6
Richtlinien für die Dimensionierung . . . 9
Performance-Optimierung . . . 11
Baustein mit hoher Kapazität . . . 13

Überprüfen des Lösungsdesigns

Designübersicht

Spezifische Geräte, Kabel und Konfigurationen sind erforderlich, um die BeeGFS auf NetApp Lösung zu unterstützen, die das parallele Filesystem BeeGFS mit den NetApp EF600 Storage-Systemen kombiniert.

Weitere Informationen:

- "Hardwarekonfiguration"
- "Softwarekonfiguration"
- "Design-Überprüfung"
- "Richtlinien für die Dimensionierung"
- "Performance-Optimierung"

Derivative Architekturen mit Variationen in Design und Performance:

- "Baustein Mit Hoher Kapazität"

Hardwarekonfiguration

Die Hardware-Konfiguration für BeeGFS auf NetApp umfasst Datei-Nodes und Netzwerkverkabelung.

Konfiguration der Datei-Nodes

File-Nodes haben zwei CPU-Sockets als separate NUMA-Zonen konfiguriert, die lokalen Zugriff auf eine gleiche Anzahl von PCIe-Steckplätzen und Arbeitsspeicher enthalten.

InfiniBand-Adapter müssen in die entsprechenden PCI-Risers oder Steckplätze gefüllt sein, damit die Workload über die verfügbaren PCIe-Lanes und Speicherkanäle ausgeglichen ist. Sie balancieren den Workload aus, indem einzelne BeeGFS-Services vollständig auf einen bestimmten NUMA-Node isoliert werden. Das Ziel besteht darin, bei jedem Datei-Node eine ähnliche Performance zu erreichen, als ob es sich um zwei unabhängige Single-Socket-Server handelte.

Die folgende Abbildung zeigt die NUMA-Konfiguration des Dateiknotens.

[]

Die BeeGFS-Prozesse sind an eine bestimmte NUMA-Zone gebunden, um sicherzustellen, dass sich die verwendeten Schnittstellen in der gleichen Zone befinden. Diese Konfiguration vermeidet den Remote-Zugriff über die Verbindung zwischen den Sockets. Die Verbindung zwischen den Sockeln wird manchmal als QPI oder GMI2 Link bezeichnet; selbst in modernen Prozessorarchitekturen können sie einen Engpass darstellen, wenn High-Speed-Netzwerke wie HDR InfiniBand eingesetzt werden.

Konfiguration der Netzwerkverkabelung

Jeder Datei-Node ist innerhalb eines Bausteins mit zwei Block-Nodes verbunden. Dabei werden insgesamt

vier redundante InfiniBand-Verbindungen verwendet. Zusätzlich verfügt jeder Datei-Node über vier redundante Verbindungen zum InfiniBand-Storage-Netzwerk.

Beachten Sie in der folgenden Abbildung Folgendes:

- Alle grün dargestellten Datei-Node-Ports werden zur Verbindung mit der Storage-Fabric verwendet. Alle anderen Datei-Node-Ports sind die direkten Verbindungen zu den Block-Nodes.
- Zwei InfiniBand-Ports in einer bestimmten NUMA-Zone werden mit den A- und B-Controllern desselben Block-Nodes verbunden.
- Ports im NUMA-Knoten 0 stellen immer eine Verbindung zum ersten Block-Knoten her.
- Die Ports im NUMA-Knoten 1 verbinden sich mit dem zweiten Block-Knoten.

[]

> Bei Speichernetzwerken mit redundanten Switches sollten die in hellgrün dargestellten Ports mit einem Switch verbunden werden. Die in Dunkelgrün-Ports sollten mit einem anderen Switch verbunden werden.

Die in der Abbildung dargestellte Verkabelungskonfiguration ermöglicht jedem BeeGFS-Dienst Folgendes:

- Laufen Sie in derselben NUMA-Zone, unabhängig davon, auf welchem Dateiknoten der BeeGFS-Service ausgeführt wird.
- Sekundäre optimale Pfade zum Front-End-Storage-Netzwerk und zu den Back-End-Block-Nodes sind vorhanden, unabhängig davon, wo ein Ausfall auftritt.
- Minimale Auswirkungen auf die Performance, wenn ein Datei-Node oder Controller in einem Block-Node gewartet werden muss

Verkabelung zur Nutzung der Bandbreite

Um die vollständige bidirektionale PCIe-Bandbreite zu nutzen, stellen Sie sicher, dass ein Port an jedem InfiniBand-Adapter mit der Storage-Fabric verbunden ist, und der andere Port mit einem Block-Node verbunden wird. Die theoretische Maximalgeschwindigkeit eines HDR InfiniBand-Ports beträgt 25 Gbit/s (nicht für Signalisierung und andere Gemeinkosten). Die maximale Bandbreite für eine Richtung in einem PCIe 4.0 x16-Steckplatz beträgt 32 Gbit/s und führt zu einem potenziellen Engpass bei der Implementierung von Datei-Nodes mit InfiniBand-Adaptern mit zwei Ports, die theoretisch 50 Gbit/s Bandbreite verarbeiten können.

Die folgende Abbildung zeigt das Verkabelungsdesign, mit dem die vollständige bidirektionale PCIe-Bandbreite genutzt werden kann.

[]

Verwenden Sie für jeden BeeGFS-Service denselben Adapter, um den bevorzugten Port für Client-Datenverkehr mit dem Pfad zu dem Block-Nodes-Controller zu verbinden, der der primäre Eigentümer dieser Service-Volumes ist. Weitere Informationen finden Sie unter "Softwarekonfiguration".

Softwarekonfiguration

Die Software-Konfiguration für BeeGFS auf NetApp umfasst BeeGFS-Netzwerkkomponenten, EF600 Block-Nodes, BeeGFS-Datei-Nodes, Ressourcengruppen und BeeGFS-Services.

BeeGFS-Netzwerkkonfiguration

Die BeeGFS-Netzwerkkonfiguration besteht aus den folgenden Komponenten.

- **Schwimmende IPs** schwimmende IPs sind eine Art virtueller IP-Adresse, die dynamisch an jeden Server im selben Netzwerk weitergeleitet werden kann. Mehrere Server können dieselbe Floating-IP-Adresse besitzen, sie kann jedoch nur auf einem Server zu einem bestimmten Zeitpunkt aktiv sein.

  Jeder BeeGFS-Serverdienst hat eine eigene IP-Adresse, die sich je nach Ausführungsort des BeeGFS-Serverdienstes zwischen Datei-Nodes verschieben kann. Diese fließende IP-Konfiguration ermöglicht jedem Service ein unabhängiges Failover auf den anderen File-Node. Der Client muss einfach die IP-Adresse für einen bestimmten BeeGFS-Service kennen. Es muss nicht wissen, welcher Datei-Node derzeit diesen Service ausführt.

- **BeeGFS-Server Multi-Homing-Konfiguration** um die Dichte der Lösung zu erhöhen, verfügt jeder Dateiknoten über mehrere Speicherschnittstellen mit IPs, die im gleichen IP-Subnetz konfiguriert sind.

  Es ist eine zusätzliche Konfiguration erforderlich, um sicherzustellen, dass diese Konfiguration wie erwartet mit dem Linux-Netzwerk-Stack funktioniert, da standardmäßig Anfragen an eine Schnittstelle auf einer anderen Schnittstelle beantwortet werden können, wenn ihre IPs im selben Subnetz sind. Neben anderen Nachteilen ist es bei diesem Standardverhalten unmöglich, RDMA-Verbindungen ordnungsgemäß einzurichten oder zu pflegen.

  Die Ansible-basierte Implementierung verarbeitet das Anziehen des Reverse Path (RP) und das ARP-Verhalten (Address Resolution Protocol) und stellt sicher, dass Floating IPs gestartet und gestoppt werden. Die entsprechenden IP-Routen und -Regeln werden dynamisch erstellt, damit die Multihomed-Netzwerkkonfiguration ordnungsgemäß funktioniert.

- **BeeGFS Client Multi-Rail-Konfiguration** *Multi-Rail* bezieht sich auf die Fähigkeit einer Anwendung, mehrere unabhängige Netzwerk-„Rails“ zu verwenden, um die Leistung zu steigern.

  BeeGFS kann zwar RDMA für die Konnektivität verwenden, doch nutzt BeeGFS IPoIB, um die Erkennung und Einrichtung von RDMA-Verbindungen zu vereinfachen. Damit BeeGFS-Clients mehrere InfiniBand-Schnittstellen verwenden können, können Sie jeden Client mit einer IP-Adresse in einem anderen Subnetz konfigurieren und dann die bevorzugten Schnittstellen für die Hälfte der BeeGFS-Serverdienste in jedem Subnetz konfigurieren.

  In der folgenden Abbildung sind hellgrün hervorgehobene Schnittstellen in einem IP-Subnetz (z. B. `100.127.0.0/16`) Und die dunkelgrünen Schnittstellen befinden sich in einem anderen Subnetz (z.B. `100.128.0.0/16`).

  Die folgende Abbildung zeigt die Verteilung des Datenverkehrs auf mehrere BeeGFS-Client-Schnittstellen.

  []

  Da jede Datei in BeeGFS normalerweise über mehrere Storage-Services verteilt wird, kann der Client dank der Multi-Rail-Konfiguration einen höheren Durchsatz erzielen als mit einem einzelnen InfiniBand-Port möglich. Die folgende Codeprobe zeigt beispielsweise eine allgemeine File-Striping-Konfiguration, die es dem Client ermöglicht, den Datenverkehr über beide Schnittstellen auszugleichen:

```
root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
Entry type: file
EntryID: 11D-624759A9-65
Metadata node: meta_01_tgt_0101 [ID: 101]
Stripe pattern details:
+ Type: RAID0
+ Chunksize: 1M
+ Number of storage targets: desired: 4; actual: 4
+ Storage targets:
  + 101 @ stor_01_tgt_0101 [ID: 101]
  + 102 @ stor_01_tgt_0101 [ID: 101]
  + 201 @ stor_02_tgt_0201 [ID: 201]
  + 202 @ stor_02_tgt_0201 [ID: 201]
```

Die Verwendung von zwei IPoIB-Subnetzen ist ein logischer Unterschied. Sie können bei Bedarf ein einzelnes physisches InfiniBand Subnetz (Speichernetzwerk) verwenden.

> In BeeGFS 7.3.0 wurde Multi-Rail-Unterstützung hinzugefügt, um die Verwendung mehrerer IB-Schnittstellen in einem IPoIB-Subnetz zu ermöglichen. Das Design für die BeeGFS auf NetApp Lösung wurde vor der allgemeinen Verfügbarkeit von BeeGFS 7.3 entwickelt und demonstriert somit den Einsatz von zwei IP-Subnetzen für die Nutzung von zwei IB-Schnittstellen auf den BeeGFS-Clients. Ein Vorteil des Ansatzes mit mehreren IP-Subnetzen besteht darin, dass Multihoming auf BeeGFS-Client-Nodes nicht mehr konfiguriert werden muss (weitere Informationen finden Sie unter "BeeGFS RDMA-Unterstützung").

Konfiguration der EF600 Block-Node

Block-Nodes bestehen aus zwei aktiv/aktiv-RAID-Controllern mit gemeinsamem Zugriff auf denselben Satz an Laufwerken. In der Regel besitzt jeder Controller die Hälfte der im System konfigurierten Volumes, jedoch kann für den anderen Controller nach Bedarf die Aufgaben übernehmen.

Multipathing-Software auf den Datei-Knoten bestimmt den aktiven und optimierten Pfad zu jedem Volume und verschiebt sich automatisch nach einem Kabel-, Adapter- oder Controller-Ausfall zum alternativen Pfad.

Das folgende Diagramm zeigt das Controller-Layout in EF600 Block-Nodes.

[]

Um eine Shared-Disk-HA-Lösung zu erleichtern, werden Volumes beiden Datei-Nodes zugeordnet, sodass sie sich bei Bedarf gegenseitig übernehmen können. Das folgende Diagramm zeigt ein Beispiel, wie BeeGFS-Service und die bevorzugte Volumeneigentümer für maximale Performance konfiguriert sind. Die Schnittstelle links von jedem BeeGFS-Dienst gibt die bevorzugte Schnittstelle an, mit der die Clients und andere Dienste Kontakt aufnehmen.

[]

Im vorherigen Beispiel kommunizieren Clients und Server Services lieber mit Storage Service 1 über Schnittstelle i1b. Storage Service 1 verwendet Schnittstelle i1a als bevorzugten Pfad zur Kommunikation mit seinen Volumes (Storage_tgt_101, 102) auf Controller A des ersten Block-Node. Diese Konfiguration nutzt die volle bidirektionale PCIe-Bandbreite, die dem InfiniBand-Adapter zur Verfügung steht, und erreicht mit einem

Dual-Port-HDR-InfiniBand-Adapter eine bessere Leistung als bei PCIe 4.0.

BeeGFS-Dateiknoten-Konfiguration

Die BeeGFS Datei-Nodes sind in einem HA-Cluster (High Availability) konfiguriert, um den Failover von BeeGFS-Services zwischen mehreren Datei-Nodes zu ermöglichen.

Das HA Cluster Design basiert auf zwei weit verbreiteten Linux HA Projekten: Corosync für Cluster-Mitgliedschaft und Pacemaker für Cluster Resource Management. Weitere Informationen finden Sie unter "Red hat Training für Add-ons mit hoher Verfügbarkeit".

NetApp hat mehrere Open Cluster Framework (OCF) Resource Agents entwickelt und erweitert, damit das Cluster die BeeGFS Ressourcen intelligent starten und überwachen kann.

BeeGFS HA Cluster

Wenn Sie einen BeeGFS-Dienst starten (mit oder ohne HA), müssen in der Regel einige Ressourcen vorhanden sein:

- IP-Adressen, in denen der Dienst erreichbar ist, werden normalerweise von Network Manager konfiguriert.
- Zugrunde liegende Dateisysteme, die als Ziele für BeeGFS zum Speichern von Daten verwendet werden.

  Diese werden typischerweise in definiert `/etc/fstab` Und montiert von systemd.

- Ein systemd-Service, der für den Start von BeeGFS-Prozessen verantwortlich ist, wenn die anderen Ressourcen bereit sind.

  Ohne zusätzliche Software werden diese Ressourcen nur auf einem einzelnen Datei-Node gestartet. Wenn der Datei-Node offline geschaltet wird, kann auf einen Teil des BeeGFS-Dateisystems zugegriffen werden.

Da mehrere Nodes jeden BeeGFS-Service starten können, muss Pacemaker sicherstellen, dass jeder Service und die abhängigen Ressourcen nur auf jeweils einem Node ausgeführt werden. Wenn beispielsweise zwei Knoten versuchen, denselben BeeGFS-Service zu starten, besteht das Risiko einer Datenbeschädigung, wenn beide versuchen, auf dieselben Dateien auf dem zugrunde liegenden Ziel zu schreiben. Um dieses Szenario zu vermeiden, setzt Pacemaker auf Corosync, um den Zustand des gesamten Clusters zuverlässig über alle Knoten hinweg zu synchronisieren und Quorum zu schaffen.

Wenn ein Fehler im Cluster auftritt, reagiert Pacemaker und startet BeeGFS-Ressourcen auf einem anderen Knoten neu. In einigen Fällen kann Pacemaker möglicherweise nicht mit dem ursprünglichen fehlerhaften Knoten kommunizieren, um zu bestätigen, dass die Ressourcen angehalten werden. Um zu überprüfen, ob der Knoten ausgefallen ist, bevor BeeGFS-Ressourcen an anderer Stelle neu gestartet werden, isoliert Pacemaker den fehlerhaften Knoten, idealerweise indem er die Stromversorgung entfernt.

Es stehen zahlreiche Open-Source-Fechten-Agenten zur Verfügung, die es Pacemaker ermöglichen, einen Knoten mit einer Stromverteilungs-Einheit (PDU) oder den Server-Baseboard-Management-Controller (BMC) mit APIs wie Redfish zu Zaun.

Wenn BeeGFS in einem HA-Cluster ausgeführt wird, werden alle BeeGFS-Services und zugrunde liegenden Ressourcen von Pacemaker in Ressourcengruppen gemanagt. Jeder BeeGFS-Service und die Ressourcen, auf die er angewiesen ist, werden in einer Ressourcengruppe konfiguriert, die sicherstellt, dass Ressourcen in der richtigen Reihenfolge gestartet und gestoppt werden und auf demselben Node zusammengelegt werden.

Für jede BeeGFS-Ressourcengruppe führt Pacemaker eine benutzerdefinierte BeeGFS-Überwachungsressource aus, die für die Erkennung von Fehlerbedingungen und die intelligente Auslösung

von Failover verantwortlich ist, wenn auf einem bestimmten Knoten kein BeeGFS-Dienst mehr verfügbar ist.

Die folgende Abbildung zeigt die Pacemaker-gesteuerten BeeGFS-Dienste und -Abhängigkeiten.

[]

> Damit mehrere BeeGFS-Dienste desselben Typs auf demselben Knoten gestartet werden, wird Pacemaker so konfiguriert, dass BeeGFS-Dienste mit der Multi-Mode-Konfigurationsmethode gestartet werden. Weitere Informationen finden Sie im "BeeGFS-Dokumentation im Multi-Modus".

Da BeeGFS-Dienste auf mehreren Nodes starten können müssen, muss die Konfigurationsdatei für jeden Dienst (normalerweise bei gefunden `/etc/beegfs`) Wird auf einem der E-Series Volumes gespeichert, die als BeeGFS-Ziel für diesen Service verwendet werden. Damit sind die Konfiguration zusammen mit den Daten für einen bestimmten BeeGFS Service für alle Nodes zugänglich, die den Service möglicherweise ausführen müssen.

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

Design-Überprüfung

Das Design der zweiten Generation für die BeeGFS auf NetApp Lösung wurde mithilfe von drei Bausteinkonfigurationsprofilen verifiziert.

Die Konfigurationsprofile umfassen Folgendes:

- Ein einzelner Baustein, einschließlich BeeGFS-Management, Metadaten und Storage-Services
- Ein BeeGFS Metadaten plus ein Storage-Baustein.
- Ein BeeGFS-Speicherbaustein.

Die Bausteine wurden mit zwei Mellanox Quantum InfiniBand-Switches (MQM8700) verbunden. Zehn BeeGFS Clients wurden auch an die InfiniBand Switches angeschlossen und zur Ausführung von Synthetic Benchmark

Utilities verwendet.

Die folgende Abbildung zeigt die BeeGFS-Konfiguration, die zur Validierung der BeeGFS auf NetApp Lösung verwendet wird.

[]

BeeGFS-Datei-Striping

Ein Vorteil paralleler Dateisysteme besteht darin, einzelne Dateien über mehrere Storage-Ziele zu verteilen, wodurch Volumes auf demselben oder verschiedenen zugrunde liegenden Storage-Systemen dargestellt werden können.

In BeeGFS können Sie Striping auf Verzeichnisbasis und pro Datei konfigurieren, um die Anzahl der für jede Datei verwendeten Ziele zu steuern und die für jeden Dateistripe verwendete Chunksize (oder Blockgröße) zu steuern. Bei dieser Konfiguration kann das Filesystem verschiedene Workload- und I/O-Profile unterstützen, ohne Services neu konfigurieren oder neu starten zu müssen. Sie können Stripe-Einstellungen mit dem anwenden `beegfs-ctl` Befehlszeilen-Tool oder Anwendungen, die die Striping-API verwenden. Weitere Informationen finden Sie in der BeeGFS-Dokumentation für "Striping" Und "Striping-API".

Um eine optimale Leistung zu erzielen, wurden während des Tests Streifenmuster angepasst und die für jeden Test verwendeten Parameter werden notiert.

IOR-Bandbreitentests: Mehrere Clients

Die IOR-Bandbreitentests verwendeten OpenMPI, um parallele Jobs des synthetischen E/A-Generatorwerkzeugs IOR auszuführen (verfügbar unter "HPC GitHub") Über alle 10 Client-Knoten zu einem oder mehreren BeeGFS-Bausteinen. Sofern nicht anders angegeben:

- Alle Tests verwendeten einen direkten I/O mit einer Übertragungsgröße von 1 MiB.
- BeeGFS-Datei-Striping wurde auf 1 MB Chunksize und ein Ziel pro Datei eingestellt.

Für IOR wurden die folgenden Parameter verwendet, wobei die Segmentanzahl angepasst wurde, um die aggregierte Dateigröße für einen Baustein auf 5 tib und 40 tib für drei Bausteine zu halten.

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

Baustein für eine BeeGFS-Basis (Management, Metadaten und Storage

Die folgende Abbildung zeigt die IOR-Testergebnisse mit einem einzelnen BeeGFS-Basisspeicher (Management, Metadaten und Storage).

[]

BeeGFS Metadaten + Storage-Baustein

Die folgende Abbildung zeigt die IOR-Testergebnisse mit einem einzelnen BeeGFS-Metadaten + einem Storage-Baustein.

[]

BeeGFS-Speicherbaustein

Die folgende Abbildung zeigt die IOR-Testergebnisse mit einem einzelnen BeeGFS-Storage-only-Baustein.

[]

Drei BeeGFS-Bausteine

Die folgende Abbildung zeigt die IOR-Testergebnisse mit drei BeeGFS-Bausteinen.

[]

Wie erwartet, ist der Performance-Unterschied zwischen dem Basis-Baustein und den nachfolgenden Metadaten + dem Storage-Baustein vernachlässigbar. Ein Vergleich zwischen Metadaten und Storage-Bausteinen und einem ausschließlich Storage-Baustein zeigt einen leichten Anstieg der Lese-Performance aufgrund der zusätzlichen Laufwerke, die als Storage-Ziele verwendet werden. Allerdings gibt es keinen wesentlichen Unterschied in der Schreib-Performance. Um eine höhere Performance zu erzielen, können Sie mehrere Bausteine gleichzeitig hinzufügen, um die Performance linear zu skalieren.

IOR-Bandbreitentests: Einzelner Client

Der IOR-Bandbreitentest nutzte OpenMPI, um mehrere IOR-Prozesse mithilfe eines einzigen leistungsstarken GPU-Servers auszuführen, um die Performance zu untersuchen, die für einen einzelnen Client erreichbar ist.

Dieser Test vergleicht auch das Verhalten und die Leistung von BeeGFS, wenn der Client so konfiguriert ist, dass er den Linux-Kernel-Page-Cache verwendet (`tuneFileCacheType = native`) Im Vergleich zum Standard `buffered` Einstellung.

Der native Caching-Modus verwendet den Linux-Kernel-Page-Cache auf dem Client, sodass die Readervorgänge nicht über das Netzwerk übertragen werden, sondern aus dem lokalen Speicher stammen.

Das folgende Diagramm zeigt die IOR-Testergebnisse mit drei BeeGFS-Bausteinen und einem einzelnen Client.

[]

> BeeGFS Striping für diese Tests wurde auf 1 MB Chunksize mit acht Zielen pro Datei eingestellt.

Obwohl die Performance bei den Schreibzugriffen und beim ersten Lesen im standardmäßigen gepufferten Modus höher ist, werden bei Workloads, die dieselben Daten mehrmals lesen, eine deutliche Performance-Steigerung im nativen Caching-Modus erzielt. Diese verbesserte Performance bei erneuten Lesevorgängen ist für Workloads wie Deep Learning wichtig, die denselben Datensatz mehrmals in vielen Epoch-Durchläufen lesen.

Metadaten-Performance-Test

Bei den Metadaten-Performance-Tests wurde das MDTest-Tool (im Rahmen von IOR enthalten) verwendet, um die Metadaten-Performance von BeeGFS zu messen. Die Tests verwendeten OpenMPI, um parallele Jobs auf allen zehn Client-Knoten auszuführen.

Die folgenden Parameter wurden verwendet, um den Benchmark-Test mit der Gesamtzahl der Prozesse von 10 auf 320 in Schritt 2 x und mit einer Dateigröße von 4k durchgeführt.

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

Die Metadaten-Performance wurde zuerst mit ein bis zwei Metadaten + Storage-Bausteinen gemessen, um die

Performance durch das Hinzufügen weiterer Bausteine zu verdeutlichen.

1 BeeGFS Metadaten + Storage-Baustein

Das folgende Diagramm zeigt die MDTest-Ergebnisse mit einer BeeGFS-Metadaten + Speicherbausteinen.

[]

Zwei BeeGFS Metadaten + Storage-Bausteine

Das folgende Diagramm zeigt die MDTest-Ergebnisse mit zwei BeeGFS-Metadaten + Speicherbausteinen.

[]

Funktionsprüfung

Im Rahmen der Validierung dieser Architektur führte NetApp mehrere Funktionstests durch, darunter:

- Ausfall eines einzelnen InfiniBand-Ports des Clients durch Deaktivieren des Switch-Ports
- Ausfall eines InfiniBand-Ports mit einem einzelnen Server durch Deaktivieren des Switch-Ports
- Sofortige Abschaltung des Servers mithilfe des BMC.
- Anmutig Platzierung eines Node im Standby-Modus und Failover-Betrieb zu einem anderen Node
- Anmutig Setzen eines Node wieder online und Failback-Services auf den ursprünglichen Node.
- Schalten Sie einen der InfiniBand-Switches mithilfe der PDU aus. Alle Tests wurden durchgeführt, während Belastungstests mit dem durchgeführt wurden `sysSessionChecksEnabled: false` Parameter auf BeeGFS-Clients gesetzt. Es wurden keine Fehler oder Störungen bei I/O festgestellt.

> Es ist ein bekanntes Problem aufgetreten (siehe "Changelog") Wenn BeeGFS-Client/Server-RDMA-Verbindungen unerwartet unterbrochen werden, entweder durch Ausfall der primären Schnittstelle (wie in definiert `connInterfacesFile`) Oder ein BeeGFS-Server fällt aus. Aktive Client-I/O kann bis zu zehn Minuten lang aufhängen, bevor der Vorgang fortgesetzt wird. Dieses Problem tritt nicht auf, wenn BeeGFS-Knoten ordnungsgemäß für geplante Wartung in den Standby-Modus versetzt oder TCP verwendet wird.

NVIDIA DGX A100 SuperPOD- und BasePOD-Validierung

NetApp validierte eine Storage-Lösung für IANVIDDGX A100 SuperPOD unter Verwendung eines ähnlichen BeeGFS Filesystem, das aus drei Bausteinen mit den Metadaten und angewandtem Storage-Konfigurationsprofil besteht. Die Qualifizierung bestand darin, die von dieser NVA beschriebene Lösung mit zwanzig DGX A100 GPU-Servern zu testen, auf denen eine Vielzahl von Storage-, Machine-Learning- und Deep-Learning-Benchmarks ausgeführt wurden. Der gesamte Storage, der für die Verwendung in NVIDIA DGX A100 SuperPOD zertifiziert ist, ist ebenfalls automatisch für die Verwendung in NVIDIA BasePOD-Architekturen zertifiziert.

Weitere Informationen finden Sie unter "NVIDIA DGX SuperPOD mit NetApp" Und "NVIDIA DGX BasePOD".

Richtlinien für die Dimensionierung

Die BeeGFS Lösung enthält Empfehlungen für das Performance- und Kapazitäts-Sizing, die auf Verifizierungstests basieren.

Die Bausteinarchitektur verfolgt das Ziel, eine Lösung zu entwickeln, die sich einfach dimensionieren lässt.

Dazu werden mehrere Bausteine hinzugefügt, die die Anforderungen für ein bestimmtes BeeGFS-System erfüllen. Mithilfe der nachstehenden Richtlinien können Sie die Anzahl und die Arten von BeeGFS-Bausteinen schätzen, die für die Anforderungen Ihrer Umgebung benötigt werden.

Beachten Sie, dass es sich bei diesen Schätzungen um die beste Performance handelt. Benchmark-Applikationen werden geschrieben und verwendet, um die Nutzung der zugrunde liegenden Filesysteme auf eine Weise zu optimieren, die reale Applikationen vielleicht nicht erreichen.

Performance-Dimensionierung

Die folgende Tabelle enthält ein empfohlene Performance-Sizing.

Konfigurationsprofil	1 MiB Lesevorgänge	1 MiB Schreibvorgänge
Metadaten + Storage	62Gibps	21Gibps
Nur Storage	64Gibps	21Gibps

Die Schätzungen zur Größe der Metadaten-Kapazität basieren auf der „Faustregel“, dass eine Kapazität von 500 GB für etwa 150 Millionen Dateien in BeeGFS ausreichend ist. (Weitere Informationen finden Sie in der BeeGFS-Dokumentation für "Systemanforderungen".)

Die Verwendung von Funktionen wie Zugriffssteuerungslisten und die Anzahl der Verzeichnisse und Dateien pro Verzeichnis wirkt sich auch darauf aus, wie schnell der Metadatenspeicherplatz verbraucht wird. Schätzungen zur Storage-Kapazität berücksichtigen neben RAID 6 und XFS-Overhead die nutzbare Laufwerkskapazität.

Kapazitätsdimensionierung für Metadaten + Storage-Bausteine

Die folgende Tabelle enthält eine empfohlene Kapazitätsdimensionierung für Metadaten und Storage-Bausteine.

Laufwerksgröße (2+2 RAID 1) Metadaten-Volume-Gruppen	Metadaten-Kapazität (Anzahl der Dateien)	Laufwerksgröße (8+2 RAID 6) Storage-Volume-Gruppen	Speicherkapazität (File-Inhalte)
1,92 TB	1,938,577,200	1,92 TB	51,77 TB
3,84 TB	3,880,388,400	3,84 TB	103,5 TB
7,68 TB	8,125,278,000	7,68 TB	216,74 TB
15,3 TB	17,269,854,000	15,3 TB	460 TB

Bei der Größenbestimmung von Metadaten und Storage-Bausteinen können die Kosten reduziert werden, da weniger Laufwerke für Metadaten-Volume-Gruppen anstelle von Storage-Volume-Gruppen verwendet werden.

Kapazitätsdimensionierung für reine Storage-Bausteine

Die folgende Tabelle zeigt die Größenanpassung der Kapazität für reine Storage-Bausteine.

Laufwerksgröße (10+2 RAID 6) Storage-Volume-Gruppen	Speicherkapazität (File-Inhalte)
1,92 TB	59,89 TB
3,84 TB	119,80 TB
7,68 TB	251,89 TB
15,3 TB	538,5 TB

Die Performance- und der Kapazitäts-Overhead, die durch das Einbinden des Management-Service in den Basis-Baustein (erster) verursacht werden, sind minimal, es sei denn, die globale Dateisperrung ist aktiviert.

Performance-Optimierung

Die BeeGFS-Lösung enthält Empfehlungen für Performance-Tuning, die auf Verifikationstests basieren.

Obwohl BeeGFS Out-of-the-Box-Performance bietet, hat NetApp eine Reihe von empfohlenen Tuning-Parametern entwickelt, um die Performance zu maximieren. Diese Parameter berücksichtigen die Funktionen der zugrunde liegenden Block-Nodes der E-Series und alle speziellen Anforderungen, die für die Ausführung von BeeGFS in einer HA-Architektur mit Shared-Festplatten erforderlich sind.

Performance-Tuning für Datei-Nodes

Zu den verfügbaren Tuning-Parametern, die Sie konfigurieren können, gehören folgende:

1. **Systemeinstellungen im UEFI/BIOS von Datei-Nodes.** um die Leistung zu maximieren, empfehlen wir die Konfiguration der Systemeinstellungen auf dem Server-Modell, das Sie als Datei-Knoten verwenden. Sie konfigurieren die Systemeinstellungen, wenn Sie die Datei-Nodes einrichten, indem Sie entweder das System-Setup (UEFI/BIOS) oder die Redfish-APIs verwenden, die vom Baseboard-Management-Controller (BMC) bereitgestellt werden.

   Die Systemeinstellungen variieren je nach Servermodell, das Sie als Dateiknoten verwenden. Die Einstellungen müssen manuell auf der Grundlage des verwendeten Servermodells konfiguriert werden. Informationen zur Konfiguration der Systemeinstellungen für die validierten Lenovo SR665-Dateiknoten finden Sie unter "Optimieren Sie die System-Einstellungen des File Node für die Performance".

2. **Standardeinstellungen für erforderliche Konfigurationsparameter.** die erforderlichen Konfigurationsparameter beeinflussen die Konfiguration von BeeGFS-Diensten und die Formatierung und Bereitstellung von E-Series-Volumes (Blockgeräte) durch Pacemaker. Die folgenden Konfigurationsparameter sind erforderlich:
   - BeeGFS Service-Konfigurationsparameter

     Sie können die Standardeinstellungen für die Konfigurationsparameter bei Bedarf überschreiben. Informationen zu den Parametern, die Sie sich für Ihre spezifischen Workloads oder Anwendungsfälle anpassen können, finden Sie im "Parameter für BeeGFS-Service-Konfiguration".

   - Für die Volume-Formatierung und die Montage-Parameter gelten die empfohlenen Standardeinstellungen, die nur für erweiterte Anwendungsfälle angepasst werden sollten. Die Standardwerte führen folgende Schritte aus:

    - Optimieren Sie die ursprüngliche Volume-Formatierung auf Basis des Zieltyps (beispielsweise Management, Metadaten oder Storage), zusammen mit der RAID-Konfiguration und Segmentgröße des zugrunde liegenden Volumes.
    - Passen Sie an, wie Pacemaker die einzelnen Lautstärkerahmen einstellt, um sicherzustellen, dass Änderungen sofort an Block-Nodes der E-Series gespült werden. So wird Datenverlust verhindert, wenn bei aktiven Schreibvorgängen Datei-Nodes ausfallen.

      Informationen zu den Parametern, die Sie sich für Ihre spezifischen Workloads oder Anwendungsfälle anpassen können, finden Sie im "Volume-Formatierung und Montage der Konfigurationsparameter".

3. **Systemeinstellungen im Linux-Betriebssystem, das auf den Dateiknoten installiert ist.** Sie können die standardmäßigen Linux-Betriebssystemeinstellungen überschreiben, wenn Sie den Ansible-Bestand in Schritt 4 von erstellen "Erstellen des Ansible-Inventars".

   Die Standardeinstellungen wurden zur Validierung der BeeGFS auf NetApp Lösung verwendet. Sie können sie jedoch an Ihre spezifischen Workloads oder Anwendungsfälle anpassen. Einige Beispiele für die Linux-Betriebssystemeinstellungen, die Sie ändern können, sind:

   - I/O-Warteschlangen an Block-Geräten der E-Series.

     Auf den als BeeGFS-Ziele verwendeten E-Series-Block-Geräten können Sie I/O-Warteschlangen folgendermaßen konfigurieren:

     - Passen Sie den Planungsalgorithmus basierend auf dem Gerätetyp (NVMe, HDD usw.) an.
     - Erhöhen Sie die Anzahl der ausstehenden Anfragen.
     - Passen Sie Anfragegrößen an.
     - Optimierung des Verhaltens beim Lesen.
   - Einstellungen für virtuellen Speicher.

     Sie können die Einstellungen für den virtuellen Speicher für eine optimale Streaming-Leistung anpassen.

   - CPU-Einstellungen.

     Sie können den CPU-Frequenzregler und andere CPU-Konfigurationen für maximale Leistung anpassen.

   - Anfragegröße lesen

     Sie können die maximale Lese-Anfraegröße für Mellanox HCAs erhöhen.

Performance-Tuning für Block-Nodes

Basierend auf den Konfigurationsprofilen, die auf einen bestimmten BeeGFS-Baustein angewendet werden, ändern sich die auf den Block-Nodes konfigurierten Volume-Gruppen leicht. Beispiel mit einem EF600 Block-Node mit 24 Laufwerken:

- Für den einzelnen Basis-Block, einschließlich BeeGFS-Management, Metadaten und Storage-Services:
  - 1x 2+2 RAID 10 Volume-Gruppe für BeeGFS Management und Metadaten-Services
  - 2 x 8+2 RAID 6-Volume-Gruppen für BeeGFS-Storage-Services

- Für BeeGFS Metadaten + Storage-Baustein:
  - 1x 2+2 RAID 10 Volume-Gruppe für BeeGFS Metadaten-Services
  - 2 x 8+2 RAID 6-Volume-Gruppen für BeeGFS-Storage-Services
- Nur für BeeGFS-Lagergebäude:
  - 2 x 10+2 RAID 6-Volume-Gruppen für BeeGFS-Storage-Services

> Da BeeGFS für Management und Metadaten im Gegensatz zu Storage erheblich weniger Speicherplatz benötigt, besteht die Möglichkeit, für die RAID 10-Volume-Gruppen kleinere Laufwerke zu verwenden. Kleinere Laufwerke sollten in den äußeren Laufwerksteckplätzen befüllt werden. Weitere Informationen finden Sie im "Implementierungsanleitungen".

Diese werden alle durch die Ansible-basierte Implementierung konfiguriert, und verschiedene andere allgemein empfohlene Einstellungen für Performance-/Verhaltensoptimierung:

- Anpassung der globalen Cache-Blockgröße auf 32 KiB und Anpassung der bedarfsorientierten Cache-Flush auf 80 %.
- Deaktivieren des automatischen Load-Balancing (sicherstellen, dass die Controller-Volume-Zuweisungen wie vorgesehen bleiben).
- Aktivieren von Lese-Caching und Deaktivieren des Read-Ahead-Caching
- Aktivieren des Schreib-Caches mit Spiegelung und Bedarf an Akku-Backups, sodass Caches während des Ausfalls eines Block-Node-Controllers bestehen.
- Festlegen der Reihenfolge, in der Laufwerke Volume-Gruppen zugewiesen werden, und Ausgleich der I/O-Vorgänge über verfügbare Laufwerkskanäle

Baustein mit hoher Kapazität

Bei der Entwicklung der BeeGFS-Standardlösung wurde auf Workloads mit hoher Performance gedenkt. Kunden, die nach Anwendungsfällen mit hoher Kapazität suchen, sollten die hier beschriebenen Design- und Performance-Unterschiede beobachten.

Hardware- und Softwarekonfiguration

Hardware- und Softwarekonfiguration für den Baustein mit hoher Kapazität ist Standard. Es sei denn, die EF600 Controller sollten durch einen EF300 Controller ersetzt werden. Sie können zwischen 1 und 7 IOM-Erweiterungsfächern mit jeweils 60 Laufwerken pro Storage-Array anschließen, Insgesamt 2 bis 14 Erweiterungsfächer pro Baustein.

Unternehmen, die ein High-Capacity-Baustein-Design implementieren, verwenden wahrscheinlich nur die Basis-Bausteinkonfiguration, die aus BeeGFS-Management, Metadaten und Storage-Services für jeden Node besteht. Um die Kosteneffizienz zu steigern, sollten Storage-Nodes mit hoher Kapazität Metadaten-Volumes auf den NVMe-Laufwerken im EF300-Controller-Gehäuse bereitstellen und Storage-Volumes für die NL-SAS-Laufwerke in den Erweiterungsfächern bereitstellen.

[]

Richtlinien für die Dimensionierung

Bei diesen Richtlinien zur Dimensionierung ist davon auszugehen, dass Bausteine mit hoher Kapazität mit einer NVMe-SSD-Volume-Gruppe von 2+2 für Metadaten im Basis-EF300-Gehäuse und 6 x 8+2 NL-SAS-

Volume-Gruppen pro IOM-Erweiterungsfach für Storage konfiguriert sind.

Laufwerkgröße (Kapazitäts-HDDs)	Kapazität pro BB (1 Fach)	Kapazität pro BB (2 Einschübe)	Kapazität pro BB (3 Einschübe)	Kapazität pro BB (4 Einschübe)
4 TB	439 TB	878 TB	1317 TB	1756 TB
8 TB	878 TB	1756 TB	2634 TB	3512 TB
10 TB	1097 TB	2195 TB	3292 TB	4390 TB
12 TB	1317 TB	2634 TB	3951 TB	5268 TB
16 TB	1756 TB	3512 TB	5268 TB	7024 TB
18 TB	1975 TB	3951 TB	5927 TB	7902 TB

Copyright-Informationen

Copyright © 2024 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

Markeninformationen

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.